La Consolacion University Philippines
Valenzuela St., Capitol View Park Subdivision, Bulihan, City of Malolos, Bulacan

Management System ISO 9001:2015
TÜVRheinland CERTIFIED
www.tuv.com ID 9108635490

SCHOOL YEAR 2025-2026
COLLEGE OF INFORMATION TECHNOLOGY AND ENGINEERING

I. COURSE CODE : IT 203

II. COURSE TITLE : Data Structure and Algorithms

III. COURSE DESCRIPTION :

This course provides a comprehensive study of the fundamental concepts, techniques, and applications of data structures and algorithms in computer science and information technology. Students will explore various data structures such as arrays, linked lists, stacks, queues, trees, and graphs, and learn how to apply them effectively to solve computational problems. The course emphasizes the design, analysis, and implementation of algorithms, including searching, sorting, and recursion, with a focus on algorithmic efficiency and complexity (Big-O notation).

Students will gain practical experience through hands-on programming assignments that require the application of different data structures and algorithms in solving real-world problems. The course also integrates the values of **problem-solving, efficiency, and adaptability**, promoting the **Marian Augustinian core values of responsibility, creativity, and community service** by encouraging collaboration and innovative thinking to solve challenges using data structures.

IV. PRE-REQUISITE : IT 102 Discrete Mathematics

V. CREDIT UNITS : **3** units

VI. COURSE LEARNING OUTCOMES:

Vision	Mission
Moved by the Spirit of the Risen Christ, La Consolacion University Philippines, envisions itself to become a catalyst for the wholistic transformational development of Catholic Augustinian Marian graduates rooted in Gospel values.	Moved by the Spirit of the Risen Christ, La Consolacion University Philippines, envisions itself to become a catalyst for the wholistic transformational development of Catholic Augustinian Marian graduates rooted in Gospel values.

EXPECTED GRADUATE ATTRIBUTES	PROGRAM OUTCOMES	COURSE LEARNING OUTCOMES
EGA 1: Critical and Creative thinker EGA 2: Competent Catholic Augustinian-Marian Professional EGA 3: Socially Responsive Steward EGA 4: Transformative Lifelong learner	PO1. Articulate and discuss the latest developments in the specific field of practice; PO2. Effectively communicate orally and in writing using both English and Filipino; PO3. Work effectively and independently in multi-disciplinary and multi-cultural teams; PO4. Act in recognition of professional, social ethical responsibility; PO5. Preserve and promote "Filipino historical and cultural heritage; PO6. Analyze complex problems, and identify and define the computing requirements needed to design and appropriate solution; PO7. Apply computing and other knowledge domains to address real-world problems; PO8. Design and develop computing solutions using a system-level perspective; PO9. Utilize modern computing tools.	CLO1: Analyze complex IT-related challenges, identify optimal data structures and algorithms to address key computing needs, and design, implement, and evaluate efficient, scalable, and innovative solutions that meet the specific requirements of real-world applications. [EGA1] [PO1, PO7, PO8] CLO2: Critically analyze and implement data structures and algorithms while demonstrating ethical responsibility, integrity, and adherence to professional standards in system administration. They will ensure the security, privacy, and compliance of IT systems through the development of efficient, secure, and ethically sound solutions that address complex computational challenges.. [EGA2] [PO3, PO4] CLO3: Design an efficient data structures and algorithms with maintenance plan that outlines critical tasks for the installation, management, and ongoing maintenance of IT systems. This plan will ensure that systems are scalable, maintainable, and optimized for performance, aligning with best practices in system management and IT infrastructure. [EGA3] [PO5]

		CLO4: Apply Augustinian principles of humility, integrity, and service in their professional IT roles, ensuring that their decisions in system management prioritize ethical responsibility, transparency, and a commitment to the common good. Through thoughtful leadership and responsible stewardship of technology, they will foster a culture of collaboration, care, and justice in managing IT systems that serve both organizational goals and societal needs. [EGA4] [PO5, PO6, PO9]

VII. LEARNING PLAN:

SCHEDULE	INTENDED LEARNING OUTCOMES	COURSE CONTENTS	SUGGESTED LEARNING ACTIVITIES	SUGGESTED ASSESSMENT TASKS	EXPECTED VALUE/S TO BE INTEGRATED
Week1	ILO1. **Recognize** their responsibilities as learners in adhering to course expectations and institutional policies. [CLO1] ILO2: **Demonstrate** commitment to academic integrity, punctuality, and active participation	**I. Orientation** A. Introduction to the Course Contents B. Course Requirements and Activities C. Course Objectives Grading System and Policies on Absences, Tardiness and Examinations	• Setting and sharing of Expectations from the Course • Discussion on the Vision, Mission and Goals of LCUP and of CITE • Presentation of Course Objectives and Contents, Course Requirements and Activities	• Statement of Expectations • Review of Course Contents, Objectives, Activities and Assessment Tools • Review of Policies on Absences, Tardiness, Examinations, Grading System	• Understanding • Open mindedness • Respect • Obedience • Interiority • Courage

	throughout the course. [CLO4]		Review of Policies on Absences, Tardiness, Examinations, Grading System		
Week 2-3	ILO1: Discuss the importance of algorithms in developing programs. [CLO1][CLO2] ILO2: Apply algorithm analysis. [CLO1][CLO2] ILO3. Demonstrate the Filipino-Catholic Augustinian Spirituality through value webbing [CLO1][CLO2]	**I. INTRODUCTION: PROBLEMS TO PROGRAMS** • Facets to Good Programs • Correctness of Program • Definition of Terms ➢ Data Type ➢ Data Structure ➢ ADTs • Properties of Algorithms	• Lecture and class discussion using power point presentation • Discovery learning **Task**: Provide students with a real-world problem (e.g., scheduling system, inventory management). They must: • Analyze the problem and break it down into smaller, manageable components. • Based on the topic discussed, design an algorithm that solves the problem. • Using programming tools like flowchart or pseudocode, visually design the	Assign a group task where students must design and implement a solution to a real-world problem (e.g., building a ticket booking system, implementing a file management system). The project should involve: • Problem analysis and specification. • Data structure selection based on problem requirements. • Algorithm design and pseudo-code development. • Program implementation and testing. • Report on the time and space complexity of the solution. **Outcome**: Students will demonstrate their ability to solve real-world problems by translating them into efficient	• Effective and Efficient management of resources • Active participation and involvement • Cooperation • Critical Thinking • Community Oriented

			step-by-step procedure	algorithms and working programs. □ Correctness of the algorithm. □ Efficient use of data structures. □ Code readability and documentation. □ Time and space complexity analysis. □ Testing and debugging completeness.	
Week 4	ILO1: Discuss the importance of algorithms in developing programs. [CLO1] [CLO2] ILO2: Determine the appropriate design principles that could be used in providing solution to a problem. [CLO1] [CLO2]	**II. Ways of Analyzing Algorithms** • Algorithm Analysis • Purpose • Factors • Time Complexity Runtime Comparisons	Conduct a lecture explaining the concepts of time and space complexity using Big-O, Big-Theta (Θ), and Big-Omega (Ω) notations. Include examples of common algorithms (e.g., searching and sorting) and walk through their complexity analysis.	**Task**: In a controlled environment, students will be given algorithm-related problems to solve under time constraints. • Problems may include: • Analyzing the complexity of a provided algorithm. • Identifying the most efficient algorithm for a given problem scenario. • Writing pseudo-code or actual code for a specified algorithm. **Assessment Criteria**: • Correctness and efficiency of the solutions.	• Critical Thinking • Active participation and involvement • Open mindedness • Courage • Humility

				• Clarity and completeness of the reasoning provided. • Ability to work under time pressure	
Week 5-6	ILO1: Be able to describe the underlying principles and step-by-step processes of Insertion Sort, Selection Sort, Merge Sort, and Quick Sort. [CLO1] [CLO2] ILO2: Compare the time and space complexities of Insertion Sort, Selection Sort, Merge Sort, and Quick Sort and classify them into best, worst, and average-case performance. [CLO1] [CLO2] ILO3: Evaluate the efficiency of different sorting algorithms by running them on datasets of varying sizes, analyzing the results, and comparing theoretical	**III. Sorting and Searching Algorithms** • Basic Terminologies: Types of Sorting Technique • Internal Sorting Algorithms • Selection Sort • Insertion Sort • Merge Sort • Quick Sort	Interactive Lecture and Algorithm Walkthroughs Introduce students to the fundamental concepts of each sorting algorithm. Activity: Deliver a lecture that explains the principles of Insertion Sort, Selection Sort, Merge Sort, and Quick Sort. Use visual aids (e.g., diagrams, flowcharts) to walk through how each algorithm processes a small dataset step-by-step. After each explanation, have students walk through the algorithm with another example	**Task**: Conduct an empirical analysis of the four sorting algorithms. **Requirements**: • Measure and compare the runtime of each algorithm using datasets of various sizes and structures. • Analyze the performance results and discuss how the empirical data aligns with theoretical complexity. • Provide graphs and tables to visualize the results. **Submission**: A written report with data analysis and performance graphs. **Assessment Criteria**: • Completeness of the performance analysis. • Clarity and accuracy of graphs and tables.	• Active participation and involvement • Critical Thinking • Appreciation • Effective and Efficient management of resources • Compassion • Courage

	and empirical performances. [CLO1] [CLO2]		dataset, either individually or in groups. **Case Study: Real-World Application of Sorting Algorithms** Analyze a case study that involves sorting data (e.g., e-commerce product sorting, search engine algorithms).	• Depth of the discussion on empirical vs. theoretical performance.	
Week 6	Review topics covered by the examination	**PRELIMINARY EXAMINATION**	Examination on the topics discussed during the Prelim Term	Result of the Examination	Skills and Competencies Critical Thinking
Week 7	ILO1: Describe the step-by-step processes of Bubble Sort, Shell Sort, Bucket Sort, and Radix Sort, and explain how each algorithm operates. [CLO1] [CLO2] ILO2: Analyze and compare the time and space complexities of Bubble Sort, Shell Sort, Bucket Sort, and Radix Sort, identifying the best, worst, and	**IV. Sorting Techniques** • Shell Sort • Quick Sort • Merge Sort • Bucket Sort **V.** Radix Sort	Introduce students to the working principles of Bubble Sort, Shell Sort, Bucket Sort, and Radix Sort. **Activity**: • Use a combination of slides and whiteboard explanations to demonstrate the step-by-step process of each algorithm. • For each algorithm, walk through how it	Students are presented with real-world sorting problems (e.g., sorting user data in an e-commerce site, organizing large log files). They must select the most appropriate sorting algorithm (from Bubble, Shell, Bucket, and Radix Sort) for each problem, explaining their rationale. **Submission**: A written report explaining the problem, chosen algorithm, and	• Active participation • Integrity • Skills and Competencies • Innovation • Courage • Humility •

	average-case scenarios for each. [CLO1] [CLO2] ILO3: Evaluate the performance of these algorithms by running them on datasets of different sizes and types (e.g., random, sorted, reverse-sorted), and comparing their execution times. [CLO1] [CLO2]		sorts a simple dataset • Use animations or algorithm visualization tools to visually illustrate the sorting steps.	justification for the selection. **Assessment Criteria**: • Correctness and justification of the algorithm chosen. • Depth of analysis on why the algorithm fits the problem. • Demonstration of knowledge about algorithm trade-offs.	
Week 8	ILO1: Be able to describe the basic principles behind common searching algorithms, such as Linear Search, Binary Search and Hashing, including their mechanisms and typical use cases. [CLO1] [CLO2] ILO2: Analyze and compare the time and space complexities of searching algorithms and explain their performance in different contexts (best case, average case, and worst case)	**VI.** Searching Algorithms • Linear Search • Binary Search • Hashing	Provide students with practical experience in implementing and testing searching algorithms. **Activity**: Students will document their findings, graphing the performance of each algorithm as the dataset size grows, and compare the efficiency of the three algorithms. Students will learn to evaluate the performance differences between search algorithms and	Perform a time complexity analysis by benchmarking the performance of Linear Search, Binary Search, and Hash Search on datasets of varying sizes • Record execution times and compare the performance of each algorithm, highlighting their differences in terms of speed and efficiency. A detailed report including: • Execution time graphs or tables for each algorithm.	• Cooperation • Skills/Competencies • Patience • Appreciation • Innovation • Community Orientedness

	[CLO2][CLO3]		relate them to time complexity.	• Discussion on time complexity (best, average, worst case). • Reflections on the performance differences. **Assessment Criteria**: • Depth of performance analysis. • Correct interpretation of time complexity. • Clear presentation of results (graphs/tables). • Quality of reflections on algorithm efficiency.	
Week 9	ILO1: Define ADT Strings and describe their properties, operations (such as concatenation, substring, comparison, and length), and importance in data structures. [CLO2] [CLO3] ILO2: Analyze the time and space complexities of various **ADT String** operations and explain the impact of	**VII. Strings** • The ADT Strings • String Processing	Reinforce students' understanding of ADT String operations by having them code and experiment with different string manipulations. Pair students and ask them to collaboratively implement advanced string manipulations like reversing a string, finding palindromes, or checking anagram strings.	• Recitation • Quiz **Assessment Criteria**: • Correctness of implementation. • Code readability and proper documentation. • Handling of edge cases (e.g., empty strings, long strings).	• Understanding and patience in learning • Creative and critical thinking • Analytical thinking • Appreciation • Effective and efficient management of resources • Problem solving skills

	string length on performance. [CLO2] [CLO3]		After coding, students will explain their logic to their partners, giving and receiving feedback on their approach. Research and write a short report comparing how strings are represented in memory in different programming languages		
Week 10	ILO1: Explain the differences between array-based and linked list implementations of **ADT List**, and identify scenarios where each implementation is more efficient. [CLO2] [CLO3] ILO2: Analyze the time and space complexities (Big-O notation) of different list operations (insertion, deletion, traversal, etc.) and explain how the	**VIII. List** • The ADT List • Implementation of List • Insertion of Elements Insertion Deletion Transversal	Begin with an interactive lecture explaining ADT List and its different types (array-based and linked list implementations). **Activity**: • Assign students to implement a list using both array-based and linked list structures. • Students will perform operations like adding, deleting, and	**Task**: Students will analyze the time complexity of list operations (insertion, deletion, searching) for both array-based and linked list implementations. **Submission**: • A written report analyzing the time complexity (Big-O notation) of each operation. **Assessment Criteria**: • Accuracy of time complexity analysis. • Depth of explanation and justification.	• Active participation and involvement • Critical Thinking • Perseverance • Courage • Interiority

	choice of list type affects performance. [CLO2] [CLO3]		searching for elements, comparing the two implementations in terms of ease of use, memory usage, and performance.	• Clarity of writing and analysis.	
Week 10-11	ILO1: Define a **stack** as an abstract data type (ADT) and explain the **Last In, First Out (LIFO)** principle. [CLO2] [CLO3] ILO2: Identify real-world applications of stacks in fields such as operating systems (e.g., managing function calls), compilers (e.g., parsing expressions), and backtracking algorithms (e.g., maze solving). [CLO2] [CLO3]	**Stacks** • The ADT of Stacks • Implementation of stacks ➢ Push ➢ Pop ➢ Top of Stack ➢ IsEmpty	Conduct an interactive lecture explaining stacks as an Abstract Data Type (ADT). Use visual aids, such as diagrams, to illustrate the Last In, First Out (LIFO) principle. Discuss real-world applications of stacks to engage students. Pair students and assign them specific stack operations (push, pop, peek) to explain to each other. Students will prepare short presentations or demonstrations to teach their peers about their assigned operation, including code examples and use cases.	**Time Complexity Analysis Report Task**: Students will analyze the time complexity of stack operations (push, pop, peek) and provide a report explaining their findings using **Big-O notation**. **Submission**: A written report that includes the analysis of time complexities for both array-based and linked list implementations. **Assessment Criteria**: • Accuracy of time complexity analysis. • Clarity of explanations and logical reasoning.	• Humility • Patience • Appreciation • Innovation • Critical Thinking • Intellectual Awareness

				• Presentation and organization of the report.	
Week 12	Review topics covered by the examination	**MIDTERM EXAMINATION**	Examination on the topics discussed during the Midterm Term	Result of the Examination	• Honesty • Integrity Critical Thinking
Week 13-14	ILO1: describe and demonstrate the fundamental operations of a queue, including **enqueue** (inserting an element), **dequeue** (removing the front element), **front** (viewing the front element), and **isEmpty** (checking if the queue is empty). [CLO2] [CLO3] ILO2: Implement a queue using both array-based and linked list-based approaches [CLO2] [CLO3] ILO3: Analyze the time and space complexity of queue operations (enqueue, dequeue, front, etc.)	**X. Queues** • The ADT Queue • Implementation of Queues ➢ Enqueue ➢ Dequeue	• Lecture and class discussion using power point presentation • Discovery learning Students will apply queues to solve common algorithmic problems, such as scheduling tasks, managing resources in computer systems, or simulating customer service scenarios**.** Students will be able to identify real-world applications of queues in fields such as operating systems (e.g., job scheduling), network traffic management, and	**Time Complexity Analysis Report Task**: Students will analyze the time complexity of queue operations (enqueue, dequeue, front) and write a report that explains the analysis using Big-O notation. **Submission**: A written report detailing the time complexity of both array-based and linked list queue implementations. **Assessment Criteria**: • Accuracy of the time complexity analysis. • Clarity and coherence of the explanations. • Quality and organization of the report.	• Creative and critical thinking • Collaboration • Integrity • Courage • Compassion

	and explain their performance using Big-O notation. [CLO2] [CLO3]		customer service systems.		
Week 15	ILO1: Define tree traversal and explain its importance in navigating and processing data in a tree data structure. [CLO2] [CLO3] ILO2: Distinguish between the three main types of tree traversals: Pre-order, In-order, and Post-order, and understand the order in which nodes are visited during each traversal. [CLO2] [CLO3]	**XI. Traversal Trees** • Preorder • In-order • Post-Order • Notation Conversion • Converting Expression to trees • Building Expression Trees • The ADT Tree • Implementation of trees • Binary Trees Implementation of trees	**Tree Traversal Visualization Tools Activity**: Use online visualization tools or tree traversal simulators to help students visualize how each traversal method works. Have them input different tree structures and observe the traversal process step-by-step. **Teaching Strategy**: Allow students to experiment with different tree structures and see how different algorithms traverse the nodes. Discuss the differences between depth-first and breadth-first traversals.	**Problem-Solving Task: Applying Traversals to Real-World Problems Task**: Students will use tree traversal algorithms to solve a real-world problem such as: • Expression tree evaluation: Using Post-order traversal to evaluate mathematical expressions. • Binary Search Tree (BST) retrieval: Using In-order traversal to retrieve nodes in sorted order. Students will give a brief oral presentation where they explain their implementation of a specific tree traversal (e.g., Pre-order or In-order). They will walk through their code and discuss how it works, along with any challenges they encountered. **Assessment Criteria**:	• Understanding and patience in learning • Creative and critical thinking • Analytical thinking • Good and active participation and involvement • Appreciation • Peer Learning

				• Clarity and depth of explanation. • Ability to articulate how the traversal works. • Problem-solving skills when addressing any issues in the code. • Presentation skills (clarity, engagement, visuals).	
Week 16-17	ILO1: Define search trees and explain their purpose in data organization, retrieval, and management. [CLO2] [[CLO3] ILO2: Implement key operations for search trees, including **insertion**, **deletion**, and **searching** for elements in both BSTs and balanced trees like AVL and Red-Black Trees. [CLO2] [[CLO3] [CLO4]	**XII. Search Trees** • AVL Search Trees • Definition • Balancing • Rotations • Sample Building of AVL • B-Trees / M-WAY Trees • Characteristics • Implementations • Inserting Items • Removing Items	Interactive lecture introducing the concept of search trees, focusing on Binary Search Trees (BST), AVL Trees, Red-Black Trees, and B-Trees. Use visual aids like diagrams and animations to demonstrate the structure and properties of each type. Engage students by asking questions throughout the lecture to assess understanding and facilitate discussions about the differences and use cases of each tree type.	**Task**: Implement a specific type of search tree (e.g., AVL Tree, Red-Black Tree) from scratch. Include methods for insertion, deletion, and searching for elements. The implementation should also support balancing the tree after modifications. **Assessment Criteria**: • Correctness and functionality of the implemented tree. • Code organization and clarity.	• Understanding and patience in learning • Creative and critical thinking • Analytical thinking • Good and active participation and involvement • Appreciation • Effective and efficient management of resources

			Students will work individually or in pairs to implement a Binary Search Tree from scratch, including operations such as insertion, deletion, and searching for elements.	• Documentation quality and thoroughness.	• Problem solving skills • Courage
Week 18	Review topics covered by the examination	**FINAL EXAMINATION**	Examination on the topics discussed during the Final Term	Result of the Examination	• Honesty • Integrity Critical Thinking

VIII. COURSE REQUIREMENTS :

1. Class attendance
2. Recitations
3. Quizzes
4. Examinations
5. Attendance

IX. REFERENCES:

A. BOOKS

Bhagvan Kommadi. (2019). *Learn Data Structures and Algorithms with Golang : Level up Your Go Programming Skills to Develop Faster and More Efficient Code*. Packt Publishing. https://search.ebscohost.com/login.aspx?direct=true&db=e000xww&AN=2094782&site=ehost-live.

Chandra Sekhar Nayak, & Rivu Chakraborty. (2019). *Hands-On Data Structures and Algorithms with Kotlin : Level up Your*

Programming Skills by Understanding How Kotlin's Data Structure Works. Packt Publishing. https://search.ebscohost.com/login.aspx?direct=true&db=e000xww&AN=2037534&site=ehost-live

Claus Matzinger. (2019). *Hands-On Data Structures and Algorithms with Rust : Learn Programming Techniques to Build Effective, Maintainable, and Readable Code in Rust 2018*. Packt Publishing. https://search.ebscohost.com/login.aspx?direct=true&db=e000xww&AN=2012512&site=ehost-live.

Fabrizio Romano, Benjamin Baka, & Dusty Phillips. (2019). *Getting Started with Python : Understand Key Data Structures and Use Python in Object-oriented Programming*. Packt Publishing. https://search.ebscohost.com/login.aspx?direct=true&db=e000xww&AN=2036726&site=ehost-live.

B. INTERNET REFERENCES

Data Structures and Algorithms | Coursera." https://www.coursera.org/specializations/data-structures-algorithms. Accessed 19 Aug. 2023.

Data Structure and Algorithms Tutorial - Tutorialspoint." https://www.tutorialspoint.com/data_structures_algorithms/index.htm. Accessed 19 Aug. 2022.

Learn Data Structures and Algorithms - Udacity. https://www.udacity.com/course/data-structures-and-algorithms-nanodegree--nd256. Accessed 19 Aug. 2022.

Learn Data Structures and Algorithms - Programiz. https://www.programiz.com/dsa. Accessed 19 Aug. 2020.

C. JOURNALS

Bae, Sammie.(2019) "Stacks and Queues." *JavaScript Data Structures and Algorithms*, 2019, pp. 163–177., doi:10.1007/978-1-4842-3988-9_12.

X. GRADING SYSTEM :

1.	Class Standing Quiz Recitation Attendance	60%
2.	Examination	40 %
3.	Term Grade	CS (60%) + TE (40%)
4.	Final Rating	Prelim Grade + Midterm Grade + Final Grade

Prepared by: **ENGR. GLAZEN CARREY M. PEREZ** Faculty/Date	Checked and Reviewed by: **EDNA F. DAYAO, DIT** Dean-Date **ROSALYN S. GALVEZ, PhD., LPT** Asst. Vice President for Academic Affairs-Date References Certified by: **DR. MARIA MERCEDEZ M. HAZ** LIC Director/Date	Approved by: **JAY MARK B. SANTOS, PhD, LPT, CSSWB** Vice President for Academic Affairs-Date

Revision Date:	
Modifications: __Learning Outcomes __Course Contents __Suggested Learning Activities __Suggested Assessment __Expected Values to be Integrated __Course Requirements __References __Grading System	